claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_34138e24-10c\agent-ae43adbf9c747d628.jsonl`
- SHA-256: `e13abcfc66d452a832020aaa74c9993d5c7e7acd8ad1cf7e61c7d6005d77e5b4`
- Source modified: `2026-06-18T01:13:51+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_34138e24-10c`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:10:52.221Z)

```
You are reviewing a small bug-fix commit in a Python repo (badminton-highlight-indexer).
Work in this git worktree (read-only ? DO NOT modify, commit, or push anything):
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
First run: `git -C "C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/ablation-schema-fix" diff origin/master..HEAD` to see the exact
change. Then read the
surrounding code in backend/eval/ablation.py (load_videos, _is_fusion_schema, litmus_config,
default_signals) and tests/test_ablation.py (the litmus tests, _golden_present, the new
regression tests, _write_learned_corpus).

CONTEXT: The fix makes ablation.load_videos() skip "windowing-only" golden_features.json files
(leaner schema: start/end + velocity stats, NO shuttle/person/audio cue dicts) so a MIXED
corpus (6 full-fusion files + 5 leaner files) loads only the 6 ablatable files instead of
crashing with KeyError: 'shuttle'. The litmus must still reproduce the recorded Gate-A numbers
(?F1?0.074, 6/6 wins, p?0.031) on exactly those 6 files.

Be adversarial: actively try to find a reason this should NOT be merged as-is. Report concrete,
verifiable findings only (cite file:line). Do not invent issues. For each finding give a
severity: must-fix (a real bug / regression / breaks the litmus), should-fix, nit, or none.
Set blocks_merge=true ONLY if you found a genuine must-fix. Return the structured object.

YOUR LENS ? correctness & semantics:
- Does _is_fusion_schema correctly classify the real schemas? (full: keys include
shuttle/person/audio; lean:
active_frame_count/chunk_index/mean_velocity/peak_velocity/track_id_count.)
- Could a genuine fusion file ever be wrongly skipped, or a lean file wrongly kept?
- Does the filter still yield EXACTLY the original n=6 fusion files so the litmus's pinned
numbers (len==6, ?F1?0.074, sign 6/0, p?0.031, CI, seg ratios) remain valid?
- Is the audio-optional tolerance preserved (a shuttle+person file with no audio still loads)?
```

2. assistant (2026-06-18T01:10:56.732Z)

I'll start by examining the exact change in this commit, then review the surrounding code.

3. user (2026-06-18T01:10:58.455Z)

```
diff --git a/backend/eval/ablation.py b/backend/eval/ablation.py
index fec3e7d..968eefd 100644
--- a/backend/eval/ablation.py
+++ b/backend/eval/ablation.py
@@ -383,16 +383,37 @@ def litmus_config() -> AblationConfig:
 # ---------------------------------------------------------------------------- corpus I/O
```

```
+def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
+    """True iff every candidate row carries the per-cue feature dicts the ablation scorer reads
+    (``shuttle`` + ``person``; ``audio`` stays optional ? older JSONs predate it).
+
+    The velocity-windowing / distill path persists a LEANER ``*_golden_features.json`` row
+    (``start``/``end`` + velocity stats, no cue dicts) for clips that never went through fusion
+    feature extraction. ``load_videos`` SKIPS those rather than crash on ``c["shuttle"]`` ? so
a
+    mixed corpus (some fusion files, some windowing-only) loads the ablatable subset cleanly.
+    An empty candidate list is not ablatable either."""
+    return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
+
+
 def load_videos(features_dir: str) -> List[VideoCandidates]:
-    """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
-    person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
-    audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
-    mirroring ``experiment._feature_column``'s tolerance."""
+    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
+    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
+    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
+    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
+
+    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
+    R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
+    ablatable subset cleanly (see ``_is_fusion_schema``)."""
     videos: List[VideoCandidates] = []
     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
         with open(path, encoding="utf-8") as f:
             d = json.load(f)
         cands = d["candidates"]
+        if not _is_fusion_schema(cands):
+            logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
+                        "(no shuttle/person cue dicts), not an ablatable fusion-feature file",
+                        os.path.basename(path))
+            continue
         intervals = [(float(c["start"]), float(c["end"])) for c in cands]
         features: Dict[str, List[float]] = {}
         for fn in SHUTTLE_COLUMNS:
diff --git a/tests/test_ablation.py b/tests/test_ablation.py
index b29ecbb..224d497 100644
--- a/tests/test_ablation.py
+++ b/tests/test_ablation.py
@@ -357,6 +357,48 @@ def test_load_videos_tolerates_missing_audio(tmp_path):
     assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not crashed


+def _write_windowing_only(d: str, name: str) -> None:
+    """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema the
+    R3/R4 corpus-growth path persists for clips that never went through fusion feature
+    extraction (``start``/``end`` + velocity stats, NO shuttle/person/audio cue dicts). The
+    ablation loader must SKIP these, never crash on ``c["shuttle"]``."""
+    cands = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
+              "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
+    data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
+            "candidates": cands, "gts": [[0.0, 5.0]]}
+    with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
+        json.dump(data, f)
+
+
+def test_is_fusion_schema_distinguishes_lean_and_full():
+    """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and rejects
+    the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
ablatable."""
+    full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
```

```
+    no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}]          # audio
optional
+    lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
+    assert ablation._is_fusion_schema(full) is True
+    assert ablation._is_fusion_schema(no_audio) is True
+    assert ablation._is_fusion_schema(lean) is False
+    assert ablation._is_fusion_schema([]) is False
+
+
+def test_load_videos_skips_windowing_only_schema(tmp_path):
+    """Mixed-corpus regression (the n=11 ``KeyError: 'shuttle'``): full-fusion files load; the
+    leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than crashed
on,
+    and the litmus path runs end-to-end over the filtered ablatable subset."""
+    d = str(tmp_path)
+    _write_learned_corpus(d, n=6)                                  # 6 full-fusion files
(v0..v5)
+    for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"):  # 3 leaner real-clip names
+        _write_windowing_only(d, name)
+    # only the fusion-schema files load ? the leaner ones never crash the loader
+    videos = ablation.load_videos(d)
+    assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
+    # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed corpus
+    report = run_ablation(videos, ablation.litmus_config())
+    assert [r.signal for r in report.rows] == ["gate_a"]
+    assert report.num_videos == 6
+
+
 # ------------------------------------------------------------------------------ the golden
litmus

 GOLDEN_DIR = os.path.join(
@@ -364,7 +406,20 @@ GOLDEN_DIR = os.path.join(


 def _golden_present() -> bool:
-    return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
+    """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
cue
+    columns). Counts only ablatable files ? leaner velocity-windowing files are skipped by
+    ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus (the n=11
+    mixed-schema bug)."""
+    n = 0
+    for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
+        try:
+            with open(p, encoding="utf-8") as f:
+                cands = json.load(f).get("candidates") or []
+        except (OSError, ValueError):
+            continue
+        if ablation._is_fusion_schema(cands):
+            n += 1
+    return n >= 6


 @pytest.mark.skipif(not _golden_present(),
```

4. user (2026-06-18T01:10:58.957Z)

```
59ae347 fix(eval): ablation litmus tolerates a mixed-schema golden corpus
```

5. assistant (2026-06-18T01:11:01.377Z)

Let me read the surrounding code in both files to understand the full context.

6. user (2026-06-18T01:11:03.285Z)

```
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383     # ---------------------------------------------------------------------------- corpus
I/O
384
385
386     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
387         """True iff every candidate row carries the per-cue feature dicts the ablation
scorer reads
388         (``shuttle`` + ``person``; ``audio`` stays optional ? older JSONs predate it).
389
390         The velocity-windowing / distill path persists a LEANER ``*_golden_features.json``
row
391         (``start``/``end`` + velocity stats, no cue dicts) for clips that never went through
fusion
392         feature extraction. ``load_videos`` SKIPS those rather than crash on
``c["shuttle"]`` ? so a
393         mixed corpus (some fusion files, some windowing-only) loads the ablatable subset
cleanly.
394         An empty candidate list is not ablatable either."""
395         return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
396
397
398     def load_videos(features_dir: str) -> List[VideoCandidates]:
399         """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a
``VideoCandidates``
400         (intervals + shuttle + person + audio columns + golden gts). Audio columns are
optional
401         (older JSONs predate the audio augment) ? a missing block defaults those columns to
0.0 so
402         the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
403
404         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the
schema the
405         R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus
loads the
406         ablatable subset cleanly (see ``_is_fusion_schema``)."""
407         videos: List[VideoCandidates] = []
408         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
409             with open(path, encoding="utf-8") as f:
410                 d = json.load(f)
411             cands = d["candidates"]
412             if not _is_fusion_schema(cands):
413                 logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
414                             "(no shuttle/person cue dicts), not an ablatable fusion-feature
file",
415                             os.path.basename(path))
416                 continue
417             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
418             features: Dict[str, List[float]] = {}
419             for fn in SHUTTLE_COLUMNS:
420                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
421             for fn in PERSON_COLUMNS:
422                 features[fn] = [float(c["person"][fn]) for c in cands]
423             for fn in AUDIO_COLUMNS:
424                 features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
425             gts = [(float(a), float(b)) for a, b in d["gts"]]
426             videos.append(VideoCandidates(name=d["name"], intervals=intervals,
427                                           features=features, gts=gts))
428         return videos
```

```
429
430
431     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
432         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
433         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
434         (distinct from the static Gate-A litmus)."""
435         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
436         return experiment.Variant(name="full_learned", feature_names=cols,
437                                   threshold=threshold, min_crossings=min_crossings)
438
439
440     # ---------------------------------------------------------------------- table
rendering
441
442
443     def format_table(report: AblationReport) -> str:
444         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
445         rows = report.sorted_rows()
446         header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
447                   f"  ·  n={report.num_videos}  ·  IoU {report.iou}"
448                   f"  ·  {'LOVO-honest' if report.honest_lovo else 'static'}"
449                   f"  ·  {report.granularity.upper()}")
450         cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
451         widths = [18, 17, 18, 14, 5, 22]
452
453         def _fmt_row(cells: Sequence[str]) -> str:
454             return "  ".join(c.ljust(w) for c, w in zip(cells, widths))
455
456         lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
457         for r in rows:
458             ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
459             sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
460             verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
461             lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
462                                    "yes" if r.ci_excludes_zero else "no", verdict]))
463         return "\n".join(lines)
464
465
466     # ---------------------------------------------------------------------- PDF
export
467
468
469     def export_pdf(report: AblationReport, path: str) -> str:
470         """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's
marginal
471         contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
472         structural-protected). Reuses the muted booktabs aesthetic of
``scratch/quality_report.py``.
473
474         Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no
GPU)."""
475         from reportlab.lib import colors
476         from reportlab.lib.enums import TA_CENTER, TA_LEFT
477         from reportlab.lib.pagesizes import A4
478         from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
479         from reportlab.lib.units import cm
```

7. user (2026-06-18T01:11:04.960Z)

```
57:from backend.eval.experiment import Variant, VideoCandidates
65:SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
67:PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
```

```
"mean_player_motion",
69:AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
"audio_hit_strength_mean")
248:def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
318:def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) -> AblationReport:
352:def default_signals() -> List[Signal]:
363:        Signal("person", "person", PERSON_COLUMNS),
364:        Signal("audio", "audio", AUDIO_COLUMNS),
375:def litmus_config() -> AblationConfig:
398:def load_videos(features_dir: str) -> List[VideoCandidates]:
399:    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
402:    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
407:    videos: List[VideoCandidates] = []
419:        for fn in SHUTTLE_COLUMNS:
421:        for fn in PERSON_COLUMNS:
423:        for fn in AUDIO_COLUMNS:
426:        videos.append(VideoCandidates(name=d["name"], intervals=intervals,
435:    cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
613:        cfg = litmus_config()
616:        cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
```

8. assistant (2026-06-18T01:11:08.737Z)

Now I see an important detail. The `SHUTTLE_COLUMNS` (line 65) includes `"duration"`, but the lean schema check uses key presence of `"shuttle"`/`"person"` dicts. Let me look more closely at the column definitions and the test helpers.

9. user (2026-06-18T01:11:10.464Z)

```
55
56      from backend.eval import experiment
57      from backend.eval.experiment import Variant, VideoCandidates
58
59      logger = logging.getLogger(__name__)
60
61      # --- The signal inventory (the things the runner ablates), grouped by producer.
---------
62      # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63      # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
64      # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.
65      SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
66                                          "active_ratio", "net_crossings")
67      PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
"mean_player_motion",
68                                         "in_court_ratio", "shuttle_player_colocation")
69      AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
"audio_hit_strength_mean")
```

10. user (2026-06-18T01:11:11.208Z)

```
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
359         return [
```

```
360             Signal("shuttle.duration", "shuttle", ("duration",)),
361             Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362             Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363             Signal("person", "person", PERSON_COLUMNS),
364             Signal("audio", "audio", AUDIO_COLUMNS),
365         ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
```

11. assistant (2026-06-18T01:11:15.474Z)

Now I have an important observation. The lean schema test helper writes candidates with `peak_velocity` and `mean_velocity` as TOP-LEVEL keys, but the real shuttle column reads them from `c["shuttle"][fn]`. The lean file's velocity stats are top-level, not in a `shuttle` dict. The `_is_fusion_schema` predicate keys on `"shuttle" in c and "person" in c`, so this distinction is sound. Let me now read the test file's helpers, especially `_write_learned_corpus`, to verify the litmus reproduces n=6 numbers.

12. user (2026-06-18T01:11:16.958Z)

```
300         assert [r.ci_low for r in r1.rows] == [r.ci_low for r in r2.rows]
301         assert [r.ci_high for r in r1.rows] == [r.ci_high for r in r2.rows]
302
303
304     # ------------------------------------------------------------------------- table +
PDF
305
306
307     def test_format_table_is_sorted_and_readable():
308         videos = _learned_videos(6)
309         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
310                                                      signals=ablation.default_signals(),
311                                                      granularity="group"))
312         out = ablation.format_table(report)
313         assert "Ablation report" in out
314         assert "?F1(full?ablated)" in out
315         assert "verdict" in out
316         # the highest-?F1 signal name appears before the lowest in the rendered order
317         top = report.sorted_rows()[0].signal
318         bottom = report.sorted_rows()[-1].signal
319         assert out.index(top) < out.index(bottom)
320
321
```

```
    def test_export_pdf_is_valid_nontrivial(tmp_path):
        """A valid PDF with at least one page is produced, and it contains the verdict
legend."""
        pytest.importorskip("reportlab")
        pypdf = pytest.importorskip("pypdf")
        videos = _learned_videos(6)
        report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
                                                     signals=ablation.default_signals(),
                                                     granularity="group"))
        out = str(tmp_path / "ablation.pdf")
        ablation.export_pdf(report, out)
        assert os.path.isfile(out)
        assert os.path.getsize(out) > 1000              # non-trivial
        reader = pypdf.PdfReader(out)
        assert len(reader.pages) > 0
        text = "".join(p.extract_text() for p in reader.pages)
        assert "Ablation" in text
        assert "clears-bar" in text                     # the verdict key rendered


    # ---------------------------------------------------------------------- corpus loader
(no GPU)


    def test_load_videos_tolerates_missing_audio(tmp_path):
        """An older feature JSON without an ``audio`` block still loads ? audio columns
default to
        0.0 (loader never crashes on a pre-audio JSON)."""
        cand = {"start": 0.0, "end": 10.0,
                "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
                "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
                "label": 1}
        data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
                "candidates": [cand], "gts": [[0.0, 10.0]]}
        with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
            json.dump(data, f)
        videos = ablation.load_videos(str(tmp_path))
        assert len(videos) == 1
        assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not
crashed


    def _write_windowing_only(d: str, name: str) -> None:
        """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema
the
        R3/R4 corpus-growth path persists for clips that never went through fusion feature
        extraction (``start``/``end`` + velocity stats, NO shuttle/person/audio cue dicts).
The
        ablation loader must SKIP these, never crash on ``c["shuttle"]``."""
        cands = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
                  "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
        data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
                "candidates": cands, "gts": [[0.0, 5.0]]}
        with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
            json.dump(data, f)


    def test_is_fusion_schema_distinguishes_lean_and_full():
        """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and
rejects
        the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
ablatable."""
        full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
        no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}]          #
```

```
audio optional
        lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
        assert ablation._is_fusion_schema(full) is True
        assert ablation._is_fusion_schema(no_audio) is True
        assert ablation._is_fusion_schema(lean) is False
        assert ablation._is_fusion_schema([]) is False


    def test_load_videos_skips_windowing_only_schema(tmp_path):
        """Mixed-corpus regression (the n=11 ``KeyError: 'shuttle'``): full-fusion files
load; the
        leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than
crashed on,
        and the litmus path runs end-to-end over the filtered ablatable subset."""
        d = str(tmp_path)
        _write_learned_corpus(d, n=6)                                  # 6 full-fusion files
(v0..v5)
        for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"):  # 3 leaner real-clip
names
            _write_windowing_only(d, name)
        # only the fusion-schema files load ? the leaner ones never crash the loader
        videos = ablation.load_videos(d)
        assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
        # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed
corpus
        report = run_ablation(videos, ablation.litmus_config())
        assert [r.signal for r in report.rows] == ["gate_a"]
        assert report.num_videos == 6


    # ------------------------------------------------------------------------- the
golden litmus

    GOLDEN_DIR = os.path.join(
        os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "output")


    def _golden_present() -> bool:
        """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the
shuttle/person cue
        columns). Counts only ablatable files ? leaner velocity-windowing files are skipped
by
        ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus
(the n=11
        mixed-schema bug)."""
        n = 0
        for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
            try:
                with open(p, encoding="utf-8") as f:
                    cands = json.load(f).get("candidates") or []
            except (OSError, ValueError):
                continue
```

13. assistant (2026-06-18T01:11:19.530Z)

Now let me read the golden litmus test itself and `_write_learned_corpus` to verify the pinned numbers are unaffected.

14. user (2026-06-18T01:11:20.962Z)

```
                continue
            if ablation._is_fusion_schema(cands):
                n += 1
        return n >= 6


```

```
424
425     @pytest.mark.skipif(not _golden_present(),
426                         reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
427     def test_litmus_reproduces_gate_a():
428         """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
429         result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
430         This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
431         where the golden substrate is absent; runs on the owner box where it is present."""
432         videos = ablation.load_videos(GOLDEN_DIR)
433         assert len(videos) == 6
434         cfg = ablation.litmus_config()
435         report = run_ablation(videos, cfg)
436         assert len(report.rows) == 1
437         row = report.rows[0]
438         assert row.signal == "gate_a"
439         # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
440         assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
441         assert row.sign_wins == 6 and row.sign_losses == 0
442         assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
443         assert row.ci_low == pytest.approx(0.042, abs=0.01)
444         assert row.ci_high == pytest.approx(0.104, abs=0.01)
445         assert row.ci_excludes_zero is True
446         # Gate-A is structural ? reported but never auto-promoted to earns_keep.
447         assert row.verdict == ablation.STRUCTURAL_PROTECTED
448         # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
449         assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
450         assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
451
452
453     @pytest.mark.skipif(not _golden_present(),
454                         reason="golden output/*_golden_features.json absent")
455     def test_litmus_report_is_well_formed():
456         """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
457         report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
458         assert isinstance(report, AblationReport)
459         assert report.num_videos == 6
460         assert report.full_set_signals == ["gate_a"]
461         assert report.label_set_hash
462         out = ablation.format_table(report)
463         assert "gate_a" in out
464
465
466     # ------------------------------------------------------------------------------ edge
cases
467
468
469     def test_min_players_gate_knob_zeroed():
470         """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
471         full = Variant(name="full", min_players=2, feature_names=["a"])
472         ablated = ablation._ablated_variant(full, (), "min_players")
473         assert ablated.min_players == 0
474         assert ablated.feature_names == ["a"]            # columns untouched
475
476
477     def test_ablated_label_falls_back_to_gate_knob():
478         """With no columns, the ablated variant name carries the gate-knob label, not
'?'."""
479         full = Variant(name="full", min_crossings=2)
480         ablated = ablation._ablated_variant(full, (), "min_crossings")
481         assert "min_crossings" in ablated.name
```

```
482
483
484     # ------------------------------------------------------------------------------
CLI
485
486
487     def _write_learned_corpus(d: str, n: int = 6) -> None:
488         for k in range(n):
489             cands = []
490             for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0),
((50.0, 51.0), 0)]:
491                 shuttle = {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS}
492                 person = {fn: 0.0 for fn in ablation.PERSON_COLUMNS}
493                 person["players_in_court"] = 4.0 if pos else 0.0
494                 person["in_court_ratio"] = 1.0 if pos else 0.0
495                 audio = {fn: 0.0 for fn in ablation.AUDIO_COLUMNS}
496                 cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
497                               "audio": audio, "label": pos})
498             data = {"name": f"v{k}", "fps": 30.0, "frame_width": 1920.0,
499                     "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
500             with open(os.path.join(d, f"v{k}_golden_features.json"), "w", encoding="utf-8")
as f:
501                 json.dump(data, f)
502
503
504     def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
505         """``main()`` runs the LOGO ablation and writes both a PDF and the structured
JSON."""
506         pytest.importorskip("reportlab")
507         pytest.importorskip("pypdf")
508         d = str(tmp_path / "feat")
```

15. user (2026-06-18T01:11:22.472Z)

```
55:def _learned_videos(n: int = 6) -> list[VideoCandidates]:
148:    videos = _learned_videos(6)
164:    videos = _learned_videos(6)
177:    videos = _learned_videos(2)
239:    videos = _learned_videos(6)
281:    videos = _learned_videos(6)
295:    videos = _learned_videos(6)
308:    videos = _learned_videos(6)
326:    videos = _learned_videos(6)
390:    _write_learned_corpus(d, n=6)                                  # 6 full-fusion files
(v0..v5)
404:GOLDEN_DIR = os.path.join(
414:    for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
432:    videos = ablation.load_videos(GOLDEN_DIR)
457:    report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
487:def _write_learned_corpus(d: str, n: int = 6) -> None:
510:    _write_learned_corpus(d)
528:    _write_learned_corpus(d)
539:    _write_learned_corpus(d, n=1)
```

16. assistant (2026-06-18T01:11:26.220Z)

Now let me verify a critical concern: the litmus loads from `GOLDEN_DIR` (`output/`) via `load_videos`. The original behavior loaded ALL files; the fix now filters to fusion-schema files. The litmus asserts `len(videos) == 6`. If the real golden corpus has exactly 6 fusion files (and possibly more lean files), the filter must yield exactly those 6. Let me check whether the golden directory exists in this worktree and whether the imports (`Sequence`, `Any`, `Dict`) are available in ablation.py.

17. user (2026-06-18T01:11:28.275Z)

```
48:import argparse
49:import glob
50:import json
51:import logging
52:import os
54:from typing import Any, Dict, List, Optional, Sequence, Tuple
155:    def as_dict(self) -> Dict[str, Any]:
195:    def as_dict(self) -> Dict[str, Any]:
211:def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
218:    overrides: Dict[str, Any] = {"feature_names": ablated_fn}
248:def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
298:def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
318:def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) -> AblationReport:
386:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
453:    def _fmt_row(cells: Sequence[str]) -> str:
506:    story: List[Any] = [
528:    table_data: List[List[Any]] = [[
```

18. user (2026-06-18T01:11:29.986Z)

```
Badminton_BXH_2_golden_features.json
GX010128_golden_features.json
GX030094_golden_features.json
adarsh_avi_singles_golden_features.json
chunks
gbaaddy_golden_features.json
kushagra_singles_golden_features.json
largetest_doubles_golden_features.json
mahadevpura_1_golden_features.json
mahadevpura_2_golden_features.json
mahadevpura_singles_golden_features.json
testlarge_short_golden_features.json
---count---
11
```

19. assistant (2026-06-18T01:11:33.695Z)

This is the critical case. The real golden directory has 11 `*_golden_features.json` files (matching the n=11 bug described in the context). The fix must classify exactly 6 of them as fusion-schema for the litmus `len(videos) == 6` to hold. Let me inspect the actual candidate schemas in these 11 real files to confirm exactly 6 are fusion and 5 are lean.

20. user (2026-06-18T01:11:38.235Z)

```
Badminton_BXH_2_golden_features.json       n_cands=4    shuttle=False person=False audio=False
keys= ['active_frame_count', 'chunk_index', 'end', 'mean_velocity', 'peak_velocity', 'start',
'track_id_count']
GX010128_golden_features.json              n_cands=15   shuttle=False person=False audio=False
keys= ['active_frame_count', 'chunk_index', 'end', 'mean_velocity', 'peak_velocity', 'start',
'track_id_count']
GX030094_golden_features.json              n_cands=30   shuttle=False person=False audio=False
keys= ['active_frame_count', 'chunk_index', 'end', 'mean_velocity', 'peak_velocity', 'start',
'track_id_count']
adarsh_avi_singles_golden_features.json    n_cands=172  shuttle=True  person=True  audio=True
keys= ['audio', 'end', 'label', 'person', 'shuttle', 'start']
gbaaddy_golden_features.json               n_cands=82   shuttle=True  person=True  audio=True
keys= ['audio', 'end', 'label', 'person', 'shuttle', 'start']
kushagra_singles_golden_features.json      n_cands=20   shuttle=True  person=True  audio=True
keys= ['audio', 'end', 'label', 'person', 'shuttle', 'start']
largetest_doubles_golden_features.json     n_cands=86   shuttle=True  person=True  audio=True
keys= ['audio', 'end', 'label', 'person', 'shuttle', 'start']
mahadevpura_1_golden_features.json         n_cands=1    shuttle=False person=False audio=False
keys= ['active_frame_count', 'chunk_index', 'end', 'mean_velocity', 'peak_velocity', 'start',
```

```
'track_id_count']
mahadevpura_2_golden_features.json         n_cands=5    shuttle=False person=False audio=False
keys= ['active_frame_count', 'chunk_index', 'end', 'mean_velocity', 'peak_velocity', 'start',
'track_id_count']
mahadevpura_singles_golden_features.json   n_cands=57   shuttle=True  person=True  audio=True
keys= ['audio', 'end', 'label', 'person', 'shuttle', 'start']
testlarge_short_golden_features.json       n_cands=14   shuttle=True  person=True  audio=True
keys= ['audio', 'end', 'label', 'person', 'shuttle', 'start']
```

21. assistant (2026-06-18T01:11:44.698Z)

This confirms the fix works exactly as intended on the real corpus:
- 6 fusion-schema files: adarsh_avi_singles, gbaaddy, kushagra_singles, largetest_doubles, mahadevpura_singles, testlarge_short ? all have shuttle+person+audio.
- 5 lean files: Badminton_BXH_2, GX010128, GX030094, mahadevpura_1, mahadevpura_2 ? all have the velocity-windowing keys, no cue dicts.

The filter yields exactly 6, matching the litmus's `len(videos) == 6`. Let me now verify the litmus actually reproduces the pinned numbers on these 6 files, and run the new regression tests. The dev container lacks numpy/torch, but let me check if these tests can run (the ablation module may have heavy imports).

22. user (2026-06-18T01:12:10.624Z)

```
.............................                                                [100%]
29 passed in 23.95s
```

23. assistant (2026-06-18T01:12:13.890Z)

All 29 tests pass, including the golden litmus (since the 6 fusion files are present in `output/`). The `_golden_present()` returned true and the litmus ran. Let me confirm the litmus actually exercised (not skipped) and reproduced the numbers.

24. user (2026-06-18T01:12:18.774Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-
fix
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 3 items

tests/test_ablation.py::test_litmus_reproduces_gate_a PASSED             [ 33%]
tests/test_ablation.py::test_load_videos_skips_windowing_only_schema PASSED [ 66%]
tests/test_ablation.py::test_is_fusion_schema_distinguishes_lean_and_full PASSED [100%]

============================== 3 passed in 1.93s ==============================
```

25. assistant (2026-06-18T01:12:23.891Z)

The litmus reproduces the pinned Gate-A numbers on exactly the 6 fusion files, and the new regression tests pass. Now let me be adversarial and probe edge cases in `_is_fusion_schema`.

Key concern: `_is_fusion_schema` uses `all(...)` over candidates. If a real fusion file had even ONE candidate row missing `shuttle`/`person`, the entire file would be silently skipped (not crashed). Let me check whether real fusion files ever have heterogeneous rows, and also probe: could a lean file partially have shuttle but still be skipped correctly? Let me also check the consumer that calls `load_videos` in `main()` and the regression runner referenced in memory.

26. user (2026-06-18T01:12:25.301Z)

```
600         ap.add_argument("--litmus", action="store_true",
601                         help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
602                              "instead of the learned full-set ablation")
603         ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
604         ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
605         args = ap.parse_args()
606
607         videos = load_videos(args.features_dir)
608         if len(videos) < 2:
609             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
610                              f"(found {len(videos)})")
611
612         if args.litmus:
613             cfg = litmus_config()
614             cfg.iou, cfg.seed = args.iou, args.seed
615         else:
616             cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
617                                  granularity=args.granularity, iou=args.iou, seed=args.seed)
618
619         report = run_ablation(videos, cfg)
620         print(format_table(report))
621
622         if args.json:
623             with open(args.json, "w", encoding="utf-8") as f:
624                 json.dump(report.as_dict(), f, indent=2)
625             print(f"\nwrote report JSON -> {args.json}")
626         if args.pdf:
627             export_pdf(report, args.pdf)
628             print(f"wrote PDF report -> {args.pdf}")
629
630
631     if __name__ == "__main__":
632         main()
633
```

27. user (2026-06-18T01:12:26.769Z)

```
backend/eval/ablation.py:392:    feature extraction. ``load_videos`` SKIPS those rather than
crash on ``c["shuttle"]`` ? so a
backend/eval/ablation.py:406:    ablatable subset cleanly (see ``_is_fusion_schema``)."""
backend/eval/ablation.py:412:        if not _is_fusion_schema(cands):
backend/eval/ablation.py:413:            logger.info("ablation.load_videos: skipping %s ?
windowing-only schema "
backend/eval/ablation.py:607:    videos = load_videos(args.features_dir)
backend/eval/fusion_audio.py:121:        videos = load_videos_with_audio(args.features_dir)
backend/eval/fusion_compare.py:91:    videos = load_videos(args.features_dir)
backend/eval/golden_fixtures.py:352:    # fusion_compare.load_videos / ablation.load_videos ?
those read c['person'] unconditionally
tests/test_ablation.py:344:def test_load_videos_tolerates_missing_audio(tmp_path):
tests/test_ablation.py:355:    videos = ablation.load_videos(str(tmp_path))
tests/test_ablation.py:373:def test_is_fusion_schema_distinguishes_lean_and_full():
tests/test_ablation.py:379:    assert ablation._is_fusion_schema(full) is True
tests/test_ablation.py:380:    assert ablation._is_fusion_schema(no_audio) is True
tests/test_ablation.py:381:    assert ablation._is_fusion_schema(lean) is False
tests/test_ablation.py:382:    assert ablation._is_fusion_schema([]) is False
tests/test_ablation.py:385:def test_load_videos_skips_windowing_only_schema(tmp_path):
tests/test_ablation.py:394:    videos = ablation.load_videos(d)
tests/test_ablation.py:411:    ``load_videos``, so counting them would trip the litmus on an un-
ablatable corpus (the n=11
tests/test_ablation.py:420:        if ablation._is_fusion_schema(cands):
tests/test_ablation.py:432:    videos = ablation.load_videos(GOLDEN_DIR)
tests/test_ablation.py:457:    report = run_ablation(ablation.load_videos(GOLDEN_DIR),
```

```
ablation.litmus_config())
tests/test_audio_features.py:58:    videos = fusion_audio.load_videos_with_audio(d)
tests/test_audio_features.py:66:def test_load_videos_requires_audio(tmp_path):
tests/test_audio_features.py:76:        fusion_audio.load_videos_with_audio(str(tmp_path))
tests/test_fusion_compare.py:37:    videos = fusion_compare.load_videos(d)
tests/test_fusion_compare.py:55:    res = fusion_compare.compare(fusion_compare.load_videos(d),
iou=0.5)
tests/test_fusion_compare.py:67:    videos = fusion_compare.load_videos(str(tmp_path))
```

28. assistant (2026-06-18T01:12:31.573Z)

This is a notable finding. There are SIBLING loaders that read `c["person"]`/`c["shuttle"]` unconditionally and were NOT fixed:
- `backend/eval/fusion_compare.py:91` calls `fusion_compare.load_videos` (a different `load_videos`).
- `backend/eval/fusion_audio.py:121` calls `load_videos_with_audio`.
- `backend/eval/golden_fixtures.py:352` has a comment explicitly noting this hazard.

The fix only patched `ablation.load_videos`. Let me check whether these siblings would crash on the same mixed corpus, and whether that's in-scope. Let me read the golden_fixtures comment and the fusion_compare loader.

29. user (2026-06-18T01:12:33.155Z)

```
340         traj_csv = fdir / "trajectory.csv"
341         labels_csv = fdir / "labels.csv"
342         prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
343
344         fps, fw = _fps_fw(prov, slug)
345         iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
346         proposal = _pinned_proposal(prov)
347
348         intervals, feature_rows = distill_local.build_candidates(
349             str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
350
351         # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
352         # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
353         # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
354         features: Dict[str, List[float]] = {
355             name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
356         }
357         gts = load_gt_intervals(str(labels_csv), strict=True)
358         vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
359
360         preds = predict(CONTROL, vc)  # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
361         sc = score(preds, gts, iou)
362
363         candidates: List[Dict[str, Any]] = []
364         for j, (s, e) in enumerate(intervals):
```

30. user (2026-06-18T01:12:34.270Z)

```
1       """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3       The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4       and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
```

```
5       no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6       designed seam): the person features are extra columns the learned LogisticRegression
weights,
7       so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9       Run (no GPU):
10          python -m backend.eval.fusion_compare --features-dir output --record
11      """
12      from __future__ import annotations
13
14      import argparse
15      import glob
16      import json
17      import os
18      from typing import Any, Dict, List
19
20      from backend.eval import experiment
21      from backend.eval.eval_stats import paired_delta_ci
22      from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25      def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26          """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27          (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28          by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29          by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30          by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31          vids = [p["video"] for p in res["variant_a"]["per_video"]]
32          return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35      def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36          """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37          feature columns + golden gts). The harness derives labels from gts itself."""
38          videos: List[experiment.VideoCandidates] = []
39          for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40              with open(path, encoding="utf-8") as f:
41                  d = json.load(f)
42              cands = d["candidates"]
43              intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44              features: Dict[str, List[float]] = {}
45              for fn in SHUTTLE_FEATURE_NAMES:
46                  features[fn] = [float(c["shuttle"][fn]) for c in cands]
47              for fn in PERSON_FEATURE_NAMES:
48                  features[fn] = [float(c["person"][fn]) for c in cands]
49              gts = [(float(a), float(b)) for a, b in d["gts"]]
50              videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                       features=features, gts=gts))
52          return videos
53
54
55      def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                  threshold: float = 0.5) -> Dict[str, Any]:
57          shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
58                                            threshold=threshold)
59          fusion = experiment.Variant(name="shuttle_person_fusion",
60                                      feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
```

31. assistant (2026-06-18T01:12:42.281Z)

This confirms `fusion_compare.load_videos` (line 35-52) still reads `c["shuttle"][fn]` and `c["person"][fn]` unconditionally and would `KeyError` on the same mixed corpus. The fix only patched `ablation.load_videos`. This is a scope observation, not a defect of this specific fix ? the commit's stated scope is the ablation litmus only, and the `golden_fixtures.py:352` comment already documents that constraint as a known design hazard. I'll flag it as should-fix/none-relative-to-this-PR since it's a pre-existing parallel issue the commit doesn't claim to address. Let me verify whether the regression runner mentioned in memory (`run_regression.py`) touches `ablation.load_videos` to confirm the litmus path is the only affected consumer.

Now let me probe a subtle correctness edge in `_is_fusion_schema`: the `all(...)` semantics. If a real fusion file had a mix (some rows with cue dicts, some without), it would be silently skipped ? but real files are homogeneous (confirmed above). Let me also double-check the `_golden_present` change handles the `.get("candidates") or []` correctly and that lint/type checks pass.

32. user (2026-06-18T01:12:46.408Z)

```
/c/Users/avidu/AppData/Local/Programs/Python/Python313/Scripts/ruff
/c/Users/avidu/AppData/Local/Programs/Python/Python313/Scripts/mypy
--- ruff on changed files ---
All checks passed!
--- mypy on ablation ---
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 1 source file
```

33. assistant (2026-06-18T01:12:53.092Z)

Ruff and mypy are clean. Let me do a final adversarial check on one subtle point: the litmus's `len(videos) == 6` now depends on the filter. Before this fix, what happened? The context says the bug was `KeyError: 'shuttle'`. But there's a subtle question ? did the litmus EVER pass before the 5 lean files were added? Let me verify the `_golden_present` gate's interaction: previously it counted all 11 files (`>= 6` ? true), so the litmus RAN and CRASHED. Now it counts only the 6 fusion files (still `>= 6` ? true), litmus runs and passes. Good. But what if a future state has, say, only 5 fusion + 5 lean = 10 files? Then `_golden_present` returns false (correctly skips) rather than running a 5-video litmus that would fail `len == 6`. This is a robustness improvement, not a defect.

Let me check one more potential issue: whether `glob` ordering or the `sorted()` matters for which 6 load, and confirm the full test suite for the eval module passes (sibling loaders unaffected by this change).

34. user (2026-06-18T01:13:07.437Z)

```
............                                                             [100%]
12 passed in 7.36s
```

35. assistant (2026-06-18T01:13:12.550Z)

All sibling tests pass. Let me do a final confirmation that the litmus would have actually crashed on master (the bug being fixed) versus passing now, by checking what `master`'s `load_videos` does on this dir ? but I can confirm logically from the diff: master had no filter, so it would `KeyError` on the lean files' missing `c["shuttle"]`. The fix is verified correct.

Let me also verify the `no_audio` tolerance path end-to-end loads (a shuttle+person file with no audio dict) is genuinely preserved in `load_videos` (not just the predicate), since the task explicitly asks about this.

36. user (2026-06-18T01:13:20.214Z)

```
loaded: 1 | audio_hit_count defaulted -> [0.0]
AUDIO-OPTIONAL TOLERANCE PRESERVED: PASS
```

37. assistant (2026-06-18T01:13:28.600Z)

All checks confirm the fix is correct. Let me compile my findings into the structured output.

Summary of my verification:
- The diff adds `_is_fusion_schema` (keys on `shuttle`+`person`, audio optional) and filters in `load_videos`.
- On the REAL golden corpus (`output/`, 11 files): exactly 6 classify as fusion (adarsh_avi_singles, gbaaddy, kushagra_singles, largetest_doubles, mahadevpura_singles, testlarge_short) and 5 as lean (Badminton_BXH_2, GX010128, GX030094, mahadevpura_1, mahadevpura_2). The filter yields exactly n=6.
- The golden litmus (`test_litmus_reproduces_gate_a`) actually RAN (not skipped) and passed ? pinned ?F1?0.074, 6/0 sign, p?0.031, CI, seg ratios all hold.
- All 29 ablation tests pass; sibling tests (fusion_compare, audio_features) pass (12); ruff + mypy clean.
- Audio-optional tolerance preserved end-to-end.
- No genuine fusion file is wrongly skipped (real files are schema-homogeneous); no lean file is wrongly kept.

One in-scope observation worth flagging: sibling loaders are NOT fixed (pre-existing, out of this commit's scope).

38. user (2026-06-18T01:13:41.306Z)

Structured output provided successfully

39. assistant (2026-06-18T01:13:51.567Z)

I've completed my adversarial correctness review of the mixed-schema ablation fix. The fix is correct and I found no blocking issues.

Key verification results:

1. **Schema classification is correct.** `_is_fusion_schema` keys on `shuttle`+`person` presence with `audio` optional, exactly matching the two real schemas in the corpus.

2. **The filter yields exactly n=6 on the REAL corpus.** I inspected all 11 files in `output/`: exactly 6 are full-fusion (adarsh_avi_singles, gbaaddy, kushagra_singles, largetest_doubles, mahadevpura_singles, testlarge_short) and 5 are lean velocity-windowing (Badminton_BXH_2, GX010128, GX030094, mahadevpura_1, mahadevpura_2). This is the n=11 KeyError bug described in the context.

3. **The litmus actually ran and reproduced the pinned numbers** ? not skipped. `test_litmus_reproduces_gate_a` PASSED with ?F1?0.074, 6/0 wins, p?0.031, CI, and seg ratios all within tolerance.

4. **No false skips/keeps.** Real fusion files are schema-homogeneous, so the `all(...)` predicate is safe; no lean file leaks through.

5. **Audio-optional tolerance preserved** end-to-end (verified a shuttle+person file with no audio dict loads with audio columns defaulted to 0.0).

6. **Full verification clean:** 29 ablation tests pass, sibling fusion_compare/audio_features tests (12) pass, ruff and mypy both clean.

The only non-blocking observations: the `all(...)` skip semantics are a latent (but strictly safer) assumption, and sibling loaders `fusion_compare.load_videos` / `fusion_audio.load_videos_with_audio` still crash on a mixed corpus ? but that's pre-existing and explicitly outside this commit's scope (the `golden_fixtures.py:352` comment already documents that hazard).

`blocks_merge=false` ? no must-fix.